Strata — BA Guide

For business analysts

strata-kb v1.18.0 · 2026-09-19

1. Who this guide is for

You write requirements. Tickets, user stories, acceptance criteria — and for large features, mission plans that break them down.

Your problem is not writing. It is **grounding**: making sure that when a ticket says "the field is 4 characters, alphanumeric", that claim comes from a real document, at a known revision, and stays verifiable months later when someone asks where it came from.

Strata gives you three things for that:

1. **Search over published knowledge**, so you can find the passage instead of remembering it.
2. **Citations that pin a version**, so a ticket records exactly what the source said the day you wrote it.
3. **A Definition-of-Ready gate**, so a ticket cannot reach a developer with a broken or missing citation.

And once your missions and tickets are ready, you can **publish the plan** to the hub, so developers and other teams find *what is planned* the same way they find what exists (§5), and **amend** a mission or ticket in place when the knowledge, the architecture, the scope or the delivery moves under it (§6).

1.1 What a BA repo does and does not do

Does	Does not
Read the hub's published knowledge	Ingest documents
Cite it, pinned to a hub commit	Summarize anything
Version tickets and missions in Git	Publish knowledge content
Gate them in CI	Push to your issue tracker
Publish its own plan (`<repo_id>-plan`) on demand	Publish drafts, or anything you did not mark ready

Your repository is strictly **read-only** against the *knowledge* base. You cite, you never contribute. That is deliberate: it keeps requirements traceable to a knowledge version without giving the requirements process a way to change the knowledge itself. The one thing you do send to the hub is your own plan: the missions and tickets you have reviewed, as one generated document. It sits next to the knowledge; it never edits it.

The assistant also never opens a ticket for you. It produces Markdown; you read it, commit it, and paste it into your tracker yourself.

---

2. Setup, once

```
pip install strata-kb
kb init --kind ba
```

2.1 Point at the hub

```
# .kb/config.yaml
kind: ba
hub: https://github.com/acme/kb-hub.git
repo_id: acme-requirements
intake: ""   # set this to publish your plan, see 2.5
```

`hub:` is what `kb ticket lint` and `kb query` resolve references against. `repo_id` is no longer just a label: it names your published document, `<repo_id>-plan`, and must match your repo's entry in the hub registry (2.5).

2.2 Connect your assistant to the hub

`.mcp.json` (Claude Code) and `.cursor/mcp.json` (Cursor) were scaffolded already wired to the hub's search and citation tools, but through two placeholders — `${STRATA_KB_HUB_URL}` and `${STRATA_KB_HTTP_TOKEN}` — that nothing has set yet.

Run `kb mcp-setup` (in your assistant: `/kb-mcp-setup`). It asks for the hub's HTTP base URL and token — ask your hub maintainer for the token — writes both into `.env`, makes sure git ignores that file, and then verifies them against the hub, so a wrong URL and a rejected token come back as different errors:

Variable	Example
`STRATA_KB_HUB_URL`	`http://kb-hub.example.com:8321`
`STRATA_KB_HTTP_TOKEN`	the hub's bearer token

A bare re-run of `kb mcp-setup` reads both values back out of `.env` and verifies again, so checking your connection later never means retyping the token. Prefer the hidden prompt or the `STRATA_KB_HTTP_TOKEN` environment variable over the command's `--token` flag — that flag leaves the token sitting in your shell history.

`.mcp.json` and `.cursor/mcp.json` read those two variables from your environment, not from `.env` directly, so load `.env` into your shell (`set -a; source .env; set +a`, or use direnv) and restart your assistant — MCP reads the environment only at startup.

2.3 Open the repository in your assistant

`ba-ticket-author` and `ba-mission-plan` are scaffolded as skills, commands and prompts for **Claude Code, GitHub Copilot and Cursor**. Whichever you use, the pipeline is the same.

2.4 Configure the CI gate

Setting	Where	What it is
`STRATA_KB_HUB`	Actions → **Variables**	The hub the gate resolves refs against
`KB_HUB_TOKEN`	Actions → **Secrets**	A read token — only needed for a private hub
`KB_FAIL_ON_STALE`	**Variables**, optional	Set to any value to make an upstream amendment fail the gate

Then, in branch protection, **require the `kb-ticket-lint` check** on the branch your tickets land on. Lint running in CI does not block a merge by itself; requiring the check is what does.

> A pull request opened **from a fork** cannot read repository secrets, so on a private hub the gate fails there with a hub-unreachable message. That is the gate refusing to go green without checking, not a network fault. Merge fork contributions through a branch in this repository instead.

2.5 Turn on plan publishing (optional, once per repo)

Publishing needs two things, one on each side:

1. **The hub operator registers your repo** in the hub's `federation/registry.yaml`: the key is your `owner/repo`, mapped to your `repo_id`. They leave `workflow:` unset, because the intake compares it to the full `job_workflow_ref`, which includes the publish tag, so a pin never matches.
2. **You set `intake:`** in `.kb/config.yaml` to the hub's intake service URL (the operator gives it to you). A repo created before this feature has no `intake:` key: add `intake: <url>` by hand, then re-run `kb init --kind ba`. That writes the `kb-publish` workflow, the `kb-publish` skill (with its Copilot and Cursor wrappers) and `.gitattributes`, and adds `.kb/*-plan/` and `.kb/index.yaml` to `.gitignore`. It never touches your `config.yaml`.

Until `intake:` is set, `kb publish` takes the snapshot route (a direct mirror or a PR on the hub, no CI or intake). Both `.kb/<repo_id>-plan/` and `.kb/index.yaml` are generated by `kb plan-ingest`, so they are gitignored: never commit them.

---

3. Writing a ticket

Invoke `/ba-ticket-author` and describe the business need. The agent runs a nine-step pipeline; your job is steps 1, 3 and 9.

```
 1 Intake            you describe the need
2 Parent mission    optional — name the mission this story belongs to
3 Ground            kb_search; YOU pick which sections apply
4 Draft             story, Given/When/Then ACs, use cases, business flow
5 Pin               kb_context_new embeds the pinned block
6 Lint              kb ticket lint until DoR: PASS
7 Ground technical  the SA fills Technical grounding; kb ticket check
8 Maturity review   two independent reviews, one round by default
9 Review → save     you read it, commit it, paste it into the tracker
```

3.1 Intake

Describe the need in business terms: the capability, who needs it, and the value. Resist writing the solution. The agent's later steps work better from a clear problem than from a pre-decided implementation.

3.2 Grounding — the step that actually matters

The agent calls `kb_search` and shows you **every candidate section**, not just its favourite. Read them. Pick the ones that genuinely apply.

This is the only step where your domain judgement cannot be delegated. The agent can find candidate passages; it cannot know that §4.12 is superseded in practice by a local policy, or that two similar sections differ in a way that matters here. Everything downstream — the draft, the pin, the developer's implementation — rests on this choice.

3.3 Drafting

The agent fills the template's **core** sections: summary, user story, background, acceptance criteria, use cases, a business-flow diagram, dependencies, out of scope, open questions. Every acceptance criterion is one Given/When/Then — a starting state, one action, one observable outcome — and every claim that touches a standard carries a `[doc-id §section]` citation.

The **extended** sections are opt-in and the agent keeps one only when the story needs it: a sequence diagram when the interaction spans more than one system (an integration, a callback), non-functional requirements when the story touches load or timing, a UI spec when there is a screen, test data when records or files are processed. Anything else is deleted. Lint warns when the story's own words call for a section that is missing; you answer with the section or `N/A — <reason>`. The code-level sequence — services, tables, messages — is the developer's to draw at design time, not the BA's to guess.

3.4 Pinning

Once you confirm the sections, the agent calls `kb_context_new` and embeds the returned `## KB context` block. That block records the hub's commit at this moment. From now on the ticket does not say "see the handbook" — it says "this is exactly what the handbook said on the day this was written, and here is how to check."

3.5 Lint

`kb ticket lint` runs, the agent fixes what it reports, and it repeats until `DoR: PASS`.

3.6 Maturity review

Once lint passes, the agent runs two independent reviews against `docs/review-rubric.md` — one scoring **Business coverage**, one scoring **Dev implementability**. It applies the fixes, re-lints, and records the round. For a ticket that is the whole review by default: what is still open is handed to you as owned questions, and a score below 4 is reported, not chased. Ask for more rounds when you want them; a ticket with more than six acceptance criteria gets them automatically, up to three rounds or until both axes reach 4.

A gap it cannot close itself becomes an owned open question, written `OPEN(<owner>)`, rather than a guess. The outcome lands in the ticket's `## Review record` section.

Read the open questions. They are the agent telling you where it knew it did not know.

3.7 Your review

The draft lands in `tickets/<folder>/<ticket-id>.md` (the folder is the parent mission, or an epic or feature slug you name). You read it, commit it, and paste it into the tracker. The assistant never does that for you. What you paste is `kb ticket export tickets/<folder>/<ticket-id>.md`: the same ticket without the template's guidance comments and without the two BA-internal sections, `## Definition of Ready` and `## Review record`. `## KB context`, `## Dependencies` and `## Technical grounding` stay — the developer reads them.

3.8 SA grounds the technical half

Step 7 runs this for you. Once lint reports `DoR: PASS`, the agent saves the draft and invokes `/sa-ticket-ground` on it as its own subagent. It fills the SA-owned `## Technical grounding` section — service, files, tables, routes,

externals, volumes, healthcheck commands and device reservations, test command — each a section id from the hub's `<repo>-code` document. It never edits your sections; a business statement that contradicts the code facts is quoted there, not corrected.

Code that does not exist yet is not missing data, it is a design decision. The SA proposes a row in the parent mission's `## Technology decisions` (status `OPEN`, a human owner) and writes `[NEW: D<n>]` pointing at it. Only code that exists and the document cannot prove — internal flow, failure modes, request bodies — goes under `Open decisions` for the developer.

`kb ticket check tickets/<ticket-id>.md` gates it: PASS only when every id resolves, every `[NEW: D<n>]` names a `DECIDED` row, and `Open decisions` is empty. A ticket with no parent mission has no table to hold a row, so the SA writes free-text `[NEW: <reason>]` instead — the gate accepts it as a note; a mission would give the decision an owner. An `OPEN` row fails and names its owner — flipping it to `DECIDED` is your call, never the agent's. The handover carries a `## Needs input` block listing exactly those rows.

The gate also compares the `Grounded on:` revision with the current `-code`. That revision moves with every dev commit, so a mismatch alone never fails:

- every cited fact still holds and no cited section changed → PASS with `[note] grounded @<old>, still valid at @<new> — cited sections unchanged`;
- a cited section changed but every cited name in it still exists → PASS with a `[warn]` and a unified diff of that section, for the developer to judge;
- a cited fact is gone (an id, a column, a route, a command, a `Files:` path, a compose fact) → FAIL, ending with `grounding drifted since @<old> (now @<new>) and no longer holds — re-run /sa-ticket-ground`.

Open decisions and undecided `[NEW: D<k>]` rows fail as before, without that line: they are your call, not a re-ground.

Run `/sa-ticket-ground` by hand only when `kb ticket check` fails with `re-run /sa-ticket-ground`.

---

4. Mission plans — for large features

A feature spanning several user stories gets a **mission plan** first. Small work goes straight to a ticket. A mission is never mandatory.

```
/ba-mission-plan
```

The pipeline is **Intake** → **Ground** → **Draft** → **Split** → **Ground services** → **Pin** → **Lint** → **Maturity review** → **Review**, and it saves `missions/M-<slug>.md`.

A mission carries:

- A **C4 Level 1** diagram (System Context) and a **C4 Level 2** diagram (Container).
- A **scope split** — what is in, what is explicitly out.
- A **user-story backlog** whose ids derive from the mission id.

```
kb mission lint missions/M-checkout-revamp.md
```

The gate enforces: required structure, both diagrams present, a well-formed backlog whose ids derive from the mission id, and every citation resolving at the pinned hub version. A `0/N US drafted` coverage warning is normal — the tickets do not exist yet.

What stays **your** judgement: whether the backlog is actually complete.

4.1 Linking tickets to a mission

When drafting a story from the backlog, name the parent mission. The agent then:

- reads `missions/<mission-id>.md` and takes the story title from the backlog row,
- writes `> Parent mission: <mission-id> @ rev <N>` on its own line directly under the ticket title, where `<N>` is the mission's current `> Revision:` (the `@ rev` part is optional; absent means rev 1),
- saves the ticket as `tickets/<mission-id>/<mission-id>-US<n>.md`.

That filename is what the back-link check looks for. Dropping the parent line is valid — the ticket just becomes untraceable to its mission. The pin matters once you amend the mission (§6.3): it is how `kb mission next` knows the ticket was written against an older revision. `kb ticket tidy` moves a flat `tickets/` into the per-mission layout by that line.

4.2 Numbering

Backlog numbering gaps are fine. If you drop a story, leave its number retired. Renumbering would break the filenames of tickets already drafted.

4.3 SA grounds the service list

Step 5 runs this for you. Once you confirm the backlog and `## Sequencing` is filled, the agent invokes `/sa-ticket-ground --mission` and it fills `## Services & order` — one row per `svc.<name>` from the hub's `<repo>-code` document, with `Depends on` copied from that record. A service the mission will create carries `[NEW: D<n>]`, pointing at a row the SA appends to `## Technology decisions` for you to decide. The gate then reports FAIL on exactly those rows until you flip them to `DECIDED` — that is the expected result, not a defect. Capability layer only: no file names, no tables, no routes — those belong to each ticket's own `## Technical grounding`, filled later at §3.8.

In a skeleton repo every row the architecture document supports cites it in the Decision cell (`[<arch-doc> §<section>]`), and the agent stops at step 5b — **Decide** — to show you the whole table once. Flip the cited rows you confirm to `DECIDED`; leave an uncited row `OPEN` with a named owner. You decide here, once, not ticket by ticket. The first story of such a mission is the **foundation slice**: what the architecture document says must exist before any feature story, with every other story depending on it in `## Sequencing`.

4.4 Which ticket next

```
kb mission next
```

Every backlog story across `missions/` comes back as one of five states:

State	Means
`done`	Its id is in a `hist.*` row of the hub's `<repo>-svc`, which `kb svc note` writes at the developer's handover and CI publishes on merge
`ready`	The ticket file exists and every `## Definition of Ready` box is ticked
`draft`	The ticket file exists but a DoR box is unticked, or there is no DoR section, or the parent mission moved on (below)
`to-draft`	No ticket yet, every `Depends on` story is `done`, every D-row that `Blocks` it is `DECIDED`
`blocked`	The reasons are named: `US <id> not done`, `D<n> OPEN (owner: <x>)`

The report ends with `Next: <us-id> — <title>`; `/ba-ticket-author` with no argument runs it first and proposes that story. A dependency that is only `draft` or `ready` still blocks: done means merged. Cross-mission order is a `Depends on` cell naming another mission's story (`M-<other>-US<n>`).

A ticket is also reported `draft`, with the reason `parent mission rev <M> affects this story`, when two things hold: its mission's `> Revision:` is higher than the ticket's `@ rev` pin, **and** a change-log row after the pin lists the story in `Affected`. Amend the ticket (§6.2) and set its pin to the mission's revision, and it is `ready` again once you re-tick.

The command is read-only and exits 0 after its report. A hub that is not configured or reachable, a `-svc` not yet published, or no mission with a `Grounded on:` line becomes a `note:` line, never a failure — nothing is marked `done` until the hub can answer.

`kb mission next --from-hub [--repo <ba-repo-id>]` computes the same report from the hub alone: no `missions/` or `tickets/` needed. A story whose ticket is published is `ready`; `to-draft` reads `not ready (no published ticket)`; the repo-only states (`draft`, the revision demotion) cannot be seen from the hub, because such tickets are not published. The hub's Plans menu (§5.3) shows the same thing in a browser.

---

5. Publishing the plan

Once a mission or ticket is reviewed, you can put it on the hub. Developers, SAs and other teams then find what is planned, not only what exists. Publishing is on demand: nothing goes out when you merge. The setup is §2.5.

5.1 What gets published

`kb plan-ingest` decides. It is deterministic, needs no LLM and no hub, and runs offline. It publishes:

Item	Published when
Mission	`kb mission lint` finds no structural error **and** every box under `## Definition of Ready` is ticked
Ticket	`kb ticket lint` finds no structural error **and** every DoR box is ticked **and** `kb mission next` would not demote it to `draft` (§4.4)

Whether a citation still resolves is **not** a publish gate: that needs the hub, and `kb-ticket-lint` already gates the pull request on it. Drafts are never published. Your `## Review record` and `## Definition of Ready` sections are not published either: being on the hub means the DoR was fully ticked.

Everything else is left out, and each one prints a line:

```
skipped: tickets/M-checkout/M-checkout-US3.md — Definition of Ready not fully ticked (6/8)
```

The reasons you can see:

Reason	What to do
`lint error: <first error>` (`(+N more)` when there are several)	Run `kb mission lint` or `kb ticket lint` on the file and fix it
`Definition of Ready not fully ticked (<ticked>/<total>)`	Finish the review, then tick the boxes. Only you tick them
`parent mission rev <M> affects this story`	The mission changed after the ticket was written: amend the ticket (§6.2)
`fenced line starting with '## ' would split the section — indent it inside the fence`	A code fence in the file holds a line that starts with `##`. Indent that line inside the fence
`id contains whitespace`	The file name has a space in it. Rename the file
`duplicate id <stem>; kept the first`	Two files share a name. Rename one
`no '# ' title line`	The file has no `# Title` line. Add one

A skip never fails the command and never blocks a publish. When it runs in GitHub Actions the same lines go to the step summary. `kb plan-ingest` exits `1` only when it cannot run: an unreadable or non-UTF-8 input file, a missing `--missions-dir` or `--tickets-dir`, an invalid repo id, a destination it did not generate, a corrupt `index.yaml`, or a write failure. Lint errors are skip reasons, not failures.

5.2 Publishing

```
/kb-publish          # or: kb publish
```

`kb publish` first runs `kb plan-ingest` and `kb build`, so what you review locally is what CI will send. Run `kb plan-ingest` yourself first to see which items ship and which are skipped. After one explicit go or no-go from you, `kb publish` tags the commit `kb-publish/<timestamp>` and pushes the tag. The tag triggers `.github/workflows/kb-publish.yml`, which runs `kb plan-ingest`, `kb build` and `kb ci-publish`; the hub's intake then opens a pull request on the hub. The content is live when that PR is merged.

`kb publish` refuses a dirty `.kb/`. Because `.kb/<repo_id>-plan/` and `.kb/index.yaml` are gitignored in a BA repo, regenerating them never makes it dirty. Do not commit them.

An item that stops qualifying (say, you un-ticked a DoR box while amending it) drops out of the plan at the next publish. That is intended: no placeholder is left, and the hub's git history keeps every earlier version.

5.3 What it looks like on the hub

The plan is one document, `<repo_id>-plan`, next to `-code` and `-svc`:

- one **group per mission** (`M-<slug>`): the mission, then its published tickets;
- a **`tickets` group** for tickets with no published parent mission;
- each section is `## <id> <title>`, with `M-<slug>-US<n>` as the ticket id.

The L3 text is your source with the title line dropped, the headings demoted one level, and the two BA-internal sections removed. `## Open questions` stays. The L2 text keeps every table byte for byte, plus the mission's summary and business goal or the ticket's story and acceptance criteria. Agents search it with `kb_search`; people read it in the web UI under **Plans**, a menu beside Documents:

Page	Shows
`/ui/plans`	Every BA repo and its missions, with counts per status. `?repo=<id>` for one repo, including its "tickets without mission"
`/ui/plans/<mission>?repo=<id>`	The mission summary and backlog: each story's status, its reasons, a link to its ticket
`/ui/plans/<mission>/<story>?repo=<id>`	The ticket itself

Status is derived and read-only: **done** (the story is in the Dev repo's `-svc` history), **ready** (its ticket is published), **not ready** (no published ticket, dependencies done) or **blocked** (with reasons). To change a status you change the source: finish and publish the ticket, or have the Dev hand it over. The tracker, not the hub, is the place for "in progress" and for assignees.

A Dev given only a ticket id fetches it from here (`kb ticket fetch <id>`); see the Developer Guide.

---

6. Keeping missions and tickets current

A mission is written once, up front, and then reality moves: cited knowledge is amended, the architecture changes, the scope shifts, delivery teaches something. This section is how you change a mission or ticket without losing track of why.

6.1 Revision and change log

Every mission and ticket carries `> Revision: <N>` directly under its title, starting at 1, and an append-only `## Change log` placed before `## Review record`:

```
| Rev | Date | Trigger | What changed | Affected | Review |
|---|---|---|---|---|---|
| 2 | 2026-10-14 | kb-context, scope | Refund window cut to 14 days | M-checkout-US3, M-checkout-US5 |
```

Column	Content
`Trigger`	One or more of `kb-context`, `architecture`, `scope`, `delivery`, comma-separated
`Affected`	In a mission, the **full** story ids (`M-<slug>-US<n>`). A bare `US3` is not matched, so a ticket would never be flagged. In a ticket, the section names. `-` when none
`Review`	The amendment's maturity scores, e.g. `B4 D4 (agent)`. Amendment reviews go here, never in `## Review record`

Lint enforces it. A missing `Revision` reads as 1 and only warns, so old files still pass. With `Revision: N` above 1, the log must hold **exactly one row per revision `2..N`, in order**, and every `Trigger` must be one of the four values; otherwise lint fails.

6.2 Amending

```
/ba-mission-plan --amend M-checkout
/ba-ticket-author --amend M-checkout-US3
```

Neither starts from Intake, creates a file, or renames anything. The agent:

1. reads the file and the `kb ticket stale --md` findings for it (and, for a mission, `kb mission next`, to see which stories are done);
2. asks you for the trigger(s) and what changed. Drift findings pre-fill the answer, but it asks rather than guesses;
3. for `kb-context`, re-pins `## KB context` with `kb_context_new` using the **full** list of refs (kept, moved and new), replacing the block. You re-confirm only the moved or new refs;
4. for a **ticket**, runs `kb ticket check` on every amend. An `architecture` trigger first hands off to `sa-ticket-ground`, which refreshes `## Technical grounding` and the `Grounded on:` revision; any other trigger calls the SA only when the check fails with `re-run /sa-ticket-ground`, and reports any other FAIL to you. For a **mission**, `architecture` **only** hands off to `sa-ticket-ground --mission` and runs the check: a mission's own stories change the `svc.*` sections it cites while it is delivered. The BA skills still never read `-code` themselves;
5. applies the change rules (§6.3), bumps `Revision` and appends the change-log row;
6. lints until PASS, and runs one maturity review of the **changed sections only**;
7. hands you the diff.

Amending a ticket also **un-ticks** the Definition of Ready box(es) whose evidence changed, and updates the `@ rev` pin once the ticket reflects every change-log row since its old pin that lists it. The agent never ticks a DoR box, and never commits or pushes. You read the diff, re-tick, commit, and `kb publish` to ship it.

6.3 Change rules

- **Done stories are frozen.** A story `kb mission next` reports `done` keeps its backlog row unchanged. New or changed work becomes a **new row with the next free number**. Numbers are never reused or renumbered: they are cited in trackers, pull requests and the Dev repo's history.
- **Stories not yet done may be edited.** List each one in the row's `Affected`.
- **Decisions are superseded, not edited.** A `DECIDED` row in `## Technology decisions` is never rewritten. Add a new row `D<n>` with `Supersedes D<k>` in its Decision cell, and set `D<k>`'s status to `SUPERSEDED`. Then `kb ticket check` rejects any `[NEW: D<k>]` that still names `D<k>`, and tells you the row that supersedes it; `kb mission next` judges a `SUPERSEDED` blocker by the row that supersedes it, so a story is not blocked for ever. If a mission or ticket still cites the old row, the amendment must carry the `architecture` trigger, so the SA re-points it. The agent lists the files that cite it.
- **Tickets follow their mission.** A ticket's parent line is `> Parent mission: M-<slug> @ rev <N>`. When the mission is amended, any ticket whose story is in the new row's `Affected` becomes `draft` in `kb mission next` (§4.4) and drops out of the published plan until you amend it.

Publishing needs nothing extra: the amended item ships on your next `kb publish`, and `Revision` plus `## Change log` make the version readable inside the document.

---

7. Using code knowledge

Besides domain documents, the hub holds two documents per product repository, published by that repository's own workflow: `<repo>-code` (generated structure — `svc.*`, `db.*`, `api.*`, `int.*`, `cmd.*`, `struct.tree`) and `<repo>-svc` (curated responsibility — what each service is actually for).

The `ba-ticket-author` and `ba-mission-plan` skills do not read either document themselves. Where a ticket or mission needs a service, table, route or file name, they write `%%TODO: verify against codebase%%` with an owned open question instead of guessing, and `/sa-ticket-ground` (§3.8, §4.3) answers those from `<repo>-code` in the SA-owned section — `## Technical grounding` in a ticket, `## Services & order` in a mission. Every id it writes is checked against the document by `kb ticket check`; what the document cannot prove — internal flow, failure modes, request bodies — is parked under `Open decisions` for the developer, who has the code.

The section reference itself, spelled the way a real citation is:

```
<repo>-code §svc.<name>
<repo>-svc §svc.<name>
```

> **One caution.** `<repo>-svc` grounds a diagram. It is never a substitute for a domain citation in an acceptance criterion. A code, format, enum or threshold that encodes a standard must still come from a pinned domain section, not from a service's responsibility text.

---

8. What the gate checks — and what it does not

8.1 Errors (the gate fails)

- **Required sections present**: Summary, User Story, Background, Acceptance Criteria, Use cases, Business flow, KB context, Definition of Ready.
- **No heading appears twice.** A second `## Technical grounding` — an SA appending instead of replacing — fails the gate; the tool reads only the first copy, so a duplicate would otherwise pass a stale section.
- **At most 10 acceptance criteria.** More than ten `- [ ]` items under `## Acceptance Criteria` fails the gate. Split the ticket — merging two conditions into one AC to fit under the cap is the thing the rule exists to stop.
- **Exactly one user story.** More than one `As a … I want … so that …` shape under `## User Story` fails the gate. One ticket per story.
- **Every `## KB context` ref resolves** at its pinned hub commit. No broken or malformed refs.
- **Every inline `[doc-id §section]` citation is backed by a pinned ref.**
- **The change log matches the revision.** With `> Revision: N` above 1, `## Change log` must hold exactly one row per revision `2..N`, in order, and every `Trigger` must be `kb-context`, `architecture`, `scope` or `delivery`.

A citation written in the old bare form still counts, with a warning telling you to bracket it.

8.2 Warnings (still your judgement)

Warning	Why it is not an error
An acceptance criterion touching a standard has no citation	The tool cannot reliably tell which criteria touch a standard
The business quality of the story	Not machine-checkable
`> Revision: <N>` line missing	It reads as 1, so a file written before revisions existed still passes
`## Review record` missing, empty, or still a placeholder	Lint can see the section, not whether the review was meaningful
A stale ref	The cited section was amended upstream. Still resolves, so not broken — but you should look.
A pinned ref the body never cites	The pin may be background the ticket did not need to quote
An acceptance criterion prescribes a shell command (`docker compose …`, `curl …`, `grep …`, or two commands chained with `&&`)	The tool flags the shape; whether the AC should instead state the observable outcome is your call — the command itself belongs in `## Test data & verification` or the plan
Acceptance criteria that are not Given/When/Then (one line naming them)	The shape is the standard; the error-level floor is still a measurable value or an owned `OPEN(<owner>)`, so an older ticket passes with this one line
The story mentions load, a screen, records, or an integration and the matching extended section is missing	A keyword hit; answer with the section or `N/A — <reason>`
An extended section is present but empty, or `## Sequence diagram` is a placeholder	Delete it, fill it, or write `N/A — <reason>`

8.3 Making staleness fail

```
kb ticket lint tickets/ABC-123.md --fail-on-stale
kb mission lint missions/M-checkout.md --fail-on-stale
kb resolve tickets/ABC-123.md --status-only
```

Or set the `KB_FAIL_ON_STALE` repository variable to make the CI gate do it for every ticket.

8.4 Why the workflow is not path-filtered

The scaffolded workflow is named `kb-ticket-lint` and triggers on every pull request, not only on changes under `tickets/` or `missions/`. That is deliberate.

GitHub never synthesises a passing status for a job that never started. A `paths:` filter on a *required* check would leave any pull request touching neither directory waiting forever for a check that will never run. Instead the job always starts, inspects the diff, and dispatches by directory — exiting cleanly with a notice when there is nothing to lint.

The workflow name also predates the mission gate. It is kept for branch-protection compatibility; renaming it would leave already-protected repositories blocked on a check that no longer exists.

---

9. When a citation goes stale

Knowledge changes. A ticket written in March against a document amended in May must not silently start meaning something else.

```
kb resolve tickets/ABC-123.md          # ok | stale | broken, per ref
kb diff hr-handbook --against <rev>    # what actually changed
```

Status	Meaning	What to do
`ok`	The pinned content still matches what is published	Nothing
`stale`	The source was amended after you cited it	Run `kb diff`; decide whether the acceptance criteria need updating
`broken`	The reference no longer resolves at all	Re-ground and re-pin with `kb context new` or `kb_context_new`

`kb diff` tells you exactly what moved: section titles, one-line summaries, the summary text, the full original, sections added and removed, and whether the document's ordering changed.

Most staleness is harmless — a typo fix upstream. Some of it is a requirement that no longer matches reality. The tool tells you which sections to look at; the judgement is yours.

9.1 The daily check

A ticket that merged weeks ago is never looked at again by the pull-request gate, though the hub keeps publishing. So `.github/workflows/kb-doctor.yml` runs `kb ticket stale` every day (and on demand) over `tickets/` and `missions/`. While anything is stale or broken it keeps **one** open issue labelled `kb-stale-citations`; when everything is current again it closes it. It needs the same `STRATA_KB_HUB` variable and `KB_HUB_TOKEN` secret as the lint gate.

The issue is a table with a `Kind` column:

Only the sections a mission or ticket **cites** are compared. A Dev merge that touches nothing it used produces no row. Not every row needs an amendment: read the change, and if the requirement still holds, amend with the new pin and say so in the change-log row.

---

10. Working without an assistant

Everything the agent does, you can do at a terminal.

```
kb query "refund eligibility window" --tags payments
kb get payments-spec 4.2 --level l2
kb context new --refs "payments-spec §4.2, payments-spec §4.3"
kb ticket lint tickets/ABC-123.md
kb mission lint missions/M-checkout.md
```

`kb context new` prints the block; paste it into the ticket under `## KB context`.

---

11. Troubleshooting

Symptom	Cause	Fix
Lint: "ref does not resolve"	The section id is wrong, or that content is not published	Re-run `kb query` and re-pin. Check the hub PR actually merged.
Lint: "citation not backed by a pinned ref"	The body cites something the KB context block does not pin	Add the ref via `kb_context_new`, or remove the citation
Lint: "missing required heading"	A required section is absent — or present only inside a fenced code block	Move the real heading outside the fence
Lint passes locally, fails in CI	`STRATA_KB_HUB` is unset in Actions variables, or the hub is private and `KB_HUB_TOKEN` is missing	Configure both; see §2.4
The gate fails on a fork PR	Forks cannot read repository secrets	Merge through a branch in this repository
The assistant cannot reach the hub	`STRATA_KB_HUB_URL` or `STRATA_KB_HTTP_TOKEN` unset or wrong	Re-run `kb mcp-setup` (`/kb-mcp-setup`) — it diagnoses which one, and a bare re-run re-verifies without retyping the token
`kb mcp-setup` fails with "token was rejected" after the hub maintainer gave you a fresh one	A bare re-run reads the *old* token straight back out of `.env` — the prompt only appears when nothing is on disk yet	`STRATA_KB_HTTP_TOKEN=<new-token> kb mcp-setup`, or delete the `STRATA_KB_HTTP_TOKEN` line from `.env` and re-run
`kb query` finds nothing	The document is not published yet, or the tags are too narrow	Drop `--tags`; check with the hub owner that the publish PR merged
`kb publish` does not tag or start CI	`intake:` is empty in `.kb/config.yaml`, so it takes the snapshot route	Set `intake:` (2.5)
The intake rejects your publish	The repo is not in the hub's `federation/registry.yaml`, or its entry sets a `workflow:` pin (the intake compares it to the full `job_workflow_ref`, which includes the tag, so it never matches)	Ask the hub operator to map your `owner/repo` to `repo_id` and to leave `workflow:` unset
`kb publish` stops with ".kb/ has uncommitted changes"	Something under `.kb/` differs from `HEAD` — typically a tracked `.kb/index.yaml` or `.kb/<repo_id>-plan/`	Commit your own edits; `git rm --cached` the generated files and re-run `kb init --kind ba` so they are gitignored
A mission or ticket is missing from the plan	It did not qualify	Run `kb plan-ingest` and read its `skipped:` line (5.1)
Lint: "change log" error after an amendment	`Revision` was bumped without a matching row, or a `Trigger` is misspelled	Add exactly one row per revision `2..N`, in order, using the four trigger values
Everything resolves `broken` after an upgrade	The ticket carries a pin from an older block format	Re-pin with `kb context new`

12. Command summary

Command	Purpose	Exit
`kb mcp-setup [--hub-url URL] [--no-verify]`	Write the hub's HTTP MCP credentials into `.env` and verify them	`0` ok, `1` no value/probe failed
`kb query <text> [--tags t]`	Search published knowledge	`0`
`kb get <doc> <section>`	Fetch one section	`0`
`kb tags`	List the published tag vocabulary	`0`
`kb context new --refs "…"`	Generate a pinned citation block	`0` ok, `1` unresolvable ref
`kb resolve <file>`	Check a ticket's refs	`0` all ok, `1` broken, `2` stale
`kb diff <doc> --against <rev>`	What an amendment changed	`0`
`kb ticket lint <file>`	Ticket Definition-of-Ready gate	`0` PASS, `1` FAIL, `2` stale
`kb mission lint <file>`	Mission Definition-of-Ready gate	`0` PASS, `1` FAIL, `2` stale
`kb ticket check <file>`	SA grounding gate: every id in `## Technical grounding` exists in the hub's `-code`; a moved `Grounded on:` revision fails only when a cited fact is gone	`0` PASS, `1` FAIL
`kb ticket export <file>`	The ticket for the tracker, without BA-internal sections	`0`
`kb ticket stale [paths]`	Every ticket and mission whose citations the hub moved, `kb-context` and `architecture` (`--md`, `--json`)	`0` current, `2` stale only, `1` broken
`kb ticket tidy`	Move flat `tickets/*.md` into `tickets/<mission-id>/` by their parent line	`0`
`kb mission next [--from-hub [--repo id]]`	Which story is done, ready, draft, to-draft or blocked	`0`
`kb plan-ingest`	Build `.kb/<repo_id>-plan/` from the ready missions and tickets; prints `skipped:` lines	`0`, `1` cannot run (unreadable input, missing dir, invalid repo id, foreign destination, corrupt index, write failure)
`kb publish`	Regenerate the plan, tag, and let CI send it to the hub	`0`, `1` plan refresh or build failed, or the intake refused

Slash commands: `/ba-ticket-author` (also `--amend <ticket>`), `/ba-mission-plan` (also `--amend <M-id>`), `/sa-ticket-ground`, `/kb-publish`.

13. Upgrading the repository

```
kb init --kind ba
```

This touches **scaffold files only** — the CI workflows (including `kb-publish.yml`), the skills, command and prompt wrappers, `.gitattributes`, and the templates under `docs/` — and overwrites one only when its content differs from the new template. It also appends `.kb/*-plan/` and `.kb/index.yaml` to `.gitignore` if they are missing, and never removes a line.

`.kb/config.yaml`, `.kb/index.yaml` and `.claude/settings.json` are never touched by a plain `kb init`. That includes `config.yaml`'s `intake:` key: a repo created before plan publishing has none, so add `intake: <url>` by hand (2.5).

> **`--force` replaces all three outright**, unmerged: your hand-edited `.kb/config.yaml` including `kind`, `repo_id` and `asset_store`; `.kb/index.yaml`; and your entire `.claude/settings.json` including hooks, permissions and model. Use it deliberately.

Your own content under `tickets/` and `missions/` is not scaffolding. `kb init` never reads, writes or overwrites anything you author there — the only thing it places in those directories is an empty `.gitkeep`, so Git can track them before your first ticket exists.